



Use Case Diagrams | Unified Modeling Language (UML)

Last Updated : 09 Feb, 2024

A Use Case Diagram is a vital tool in system design, it provides a visual representation of how users interact with a system. It serves as a blueprint for understanding the functional requirements of a system from a user's perspective, aiding in the communication between stakeholders and guiding the development process.



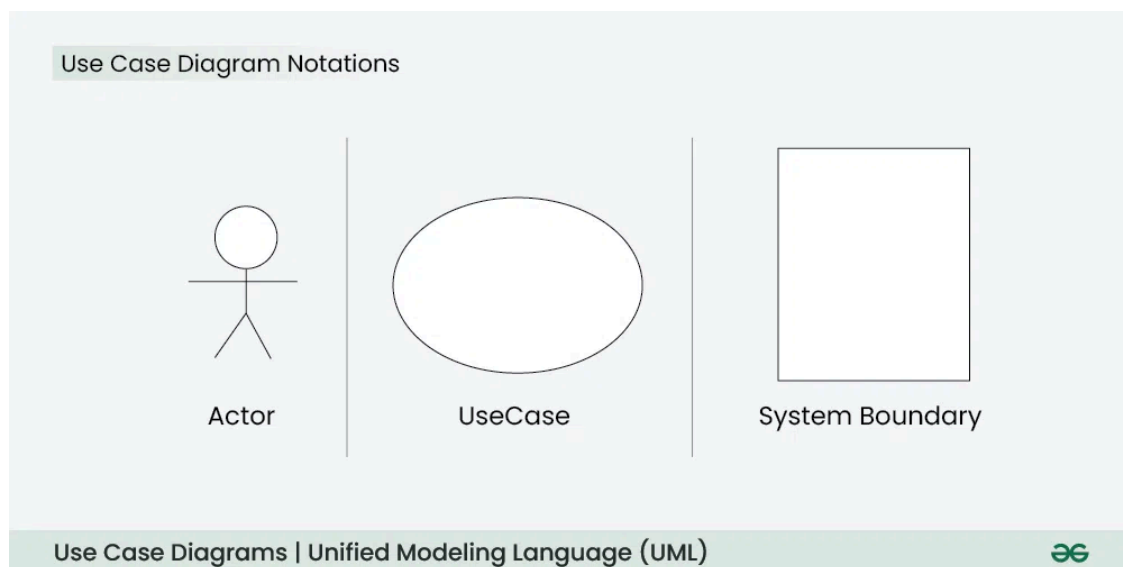
Important Topics for the Use Case Diagrams

- [What is a Use Case Diagram in UML?](#)
- [Use Case Diagram Notations](#)
- [Use Case Diagram Relationships](#)
- [How to draw a Use Case diagram in UML?](#)
- [What are common Use Case Diagram Tools and Platforms?](#)
- [What are Common Mistakes and Pitfalls while making Use Case Diagram?](#)
- [What can be Use Case Diagram Best Practices?](#)
- [What are the Purpose and Benefits of Use Case Diagrams?](#)

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

Got It !

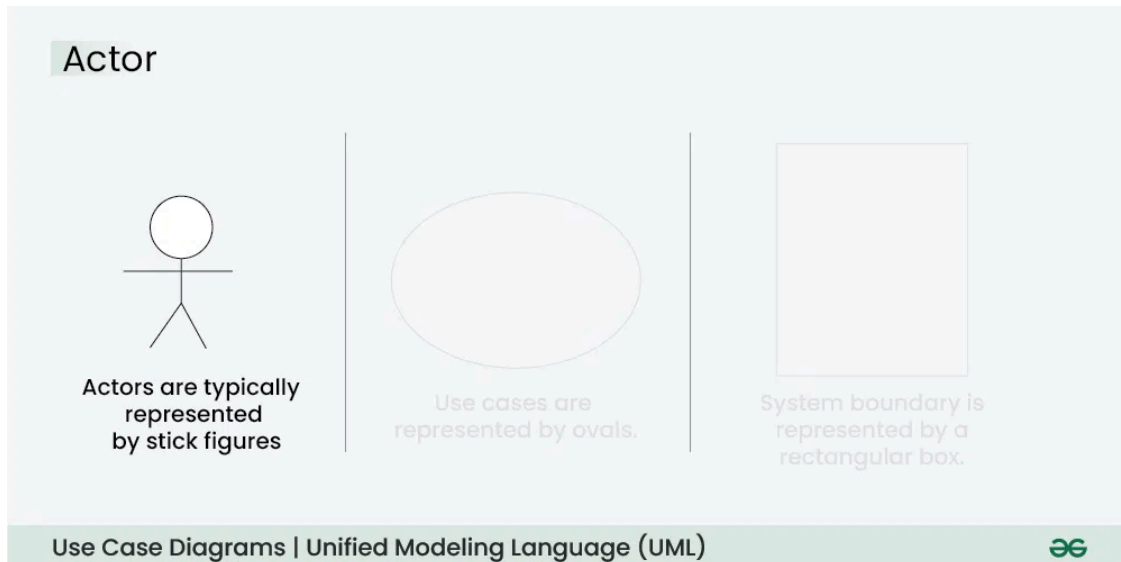
A Use Case Diagram is a type of Unified Modeling Language (UML) diagram that represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals. It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.



2. Use Case Diagram Notations

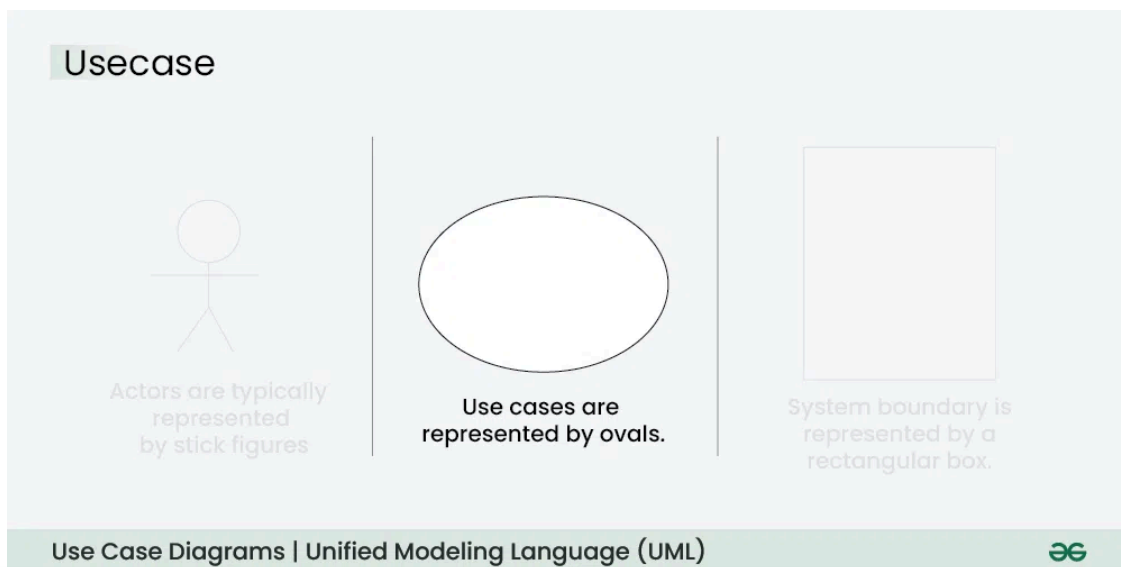
UML notations provide a visual language that enables software developers, designers, and other stakeholders to communicate and document system designs, architectures, and behaviors in a consistent and understandable manner.

Actors are external entities that interact with the system. These can include users, other systems, or hardware devices. In the context of a Use Case Diagram, actors initiate use cases and receive the outcomes. Proper identification and understanding of actors are crucial for accurately modeling system behavior.



1.2. Use Cases

Use cases are like scenes in the play. They represent specific things your system can do. In the online shopping system, examples of use cases could be “Place Order,” “Track Delivery,” or “Update Product Information”. Use cases are represented by ovals.

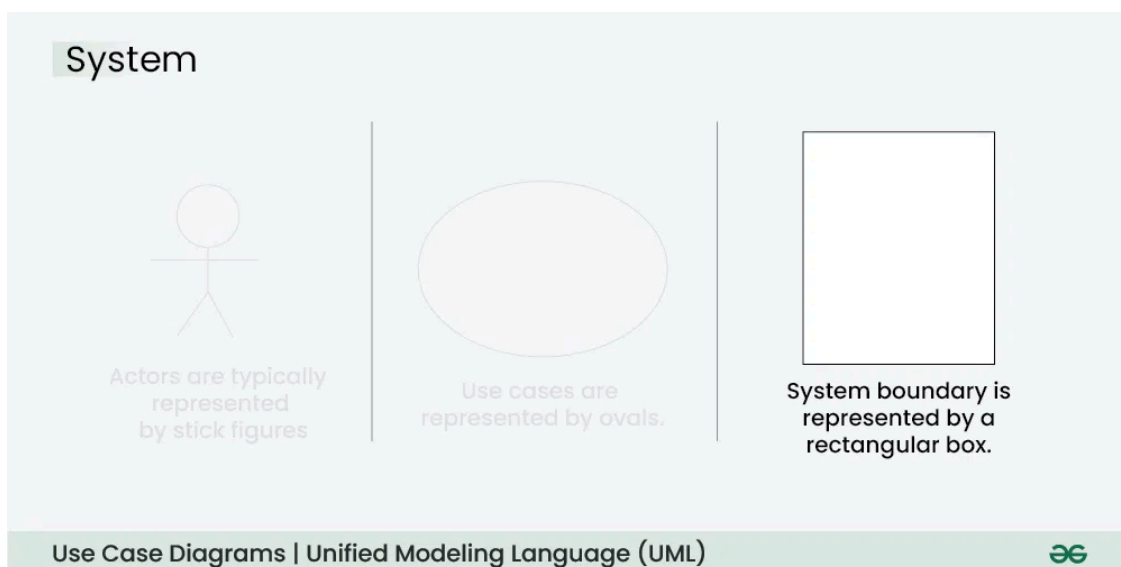


We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

The system boundary is a visual representation of the scope or limits of the system you are modeling. It defines what is inside the system and what is outside. The boundary helps to establish a clear distinction between the elements that are part of the system and those that are external to it. The system boundary is typically represented by a rectangular box that surrounds all the use cases of the system.

Purpose of System Boundary:

- **Scope Definition:** It clearly outlines the boundaries of the system, indicating which components are internal to the system and which are external actors or entities interacting with the system.
- **Focus on Relevance:** By delineating the system's scope, the diagram can focus on illustrating the essential functionalities provided by the system without unnecessary details about external entities.



3. Use Case Diagram Relationships

In a Use Case Diagram, relationships play a crucial role in depicting the interactions between actors and use cases. These relationships provide a comprehensive view of the system's functionality and its various scenarios. Let's delve into the key types of relationships and explore examples to illustrate their usage.

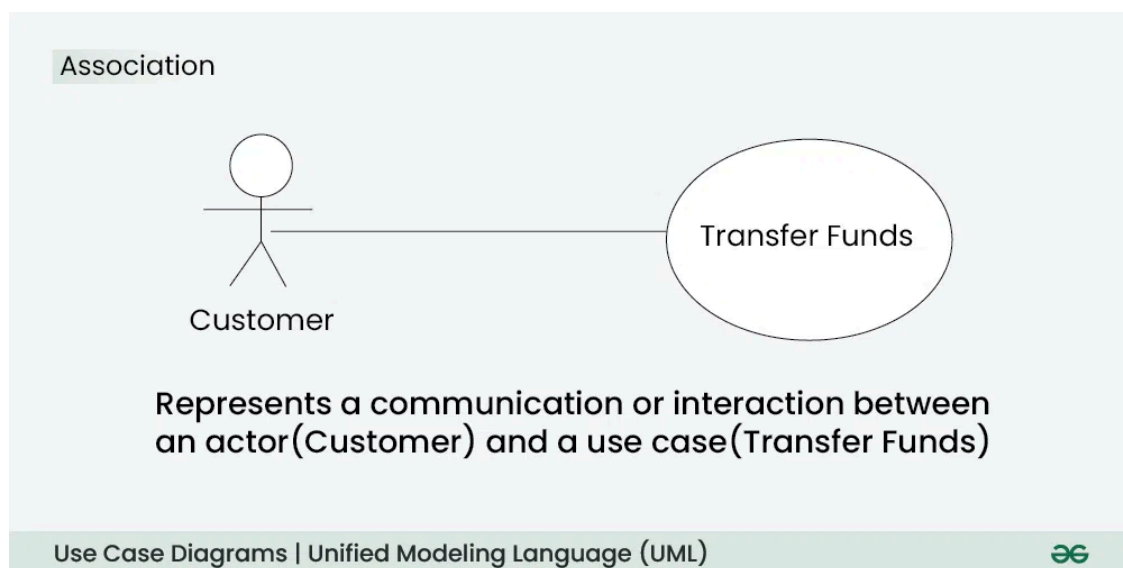
3.1. Association Relationship

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

The Association Relationship represents a communication or interaction between an actor and a use case. It is depicted by a line connecting the actor to the use case. This relationship signifies that the actor is involved in the functionality described by the use case.

Example: Online Banking System

- **Actor:** Customer
- **Use Case:** Transfer Funds
- **Association:** A line connecting the “Customer” actor to the “Transfer Funds” use case, indicating the customer’s involvement in the funds transfer process.

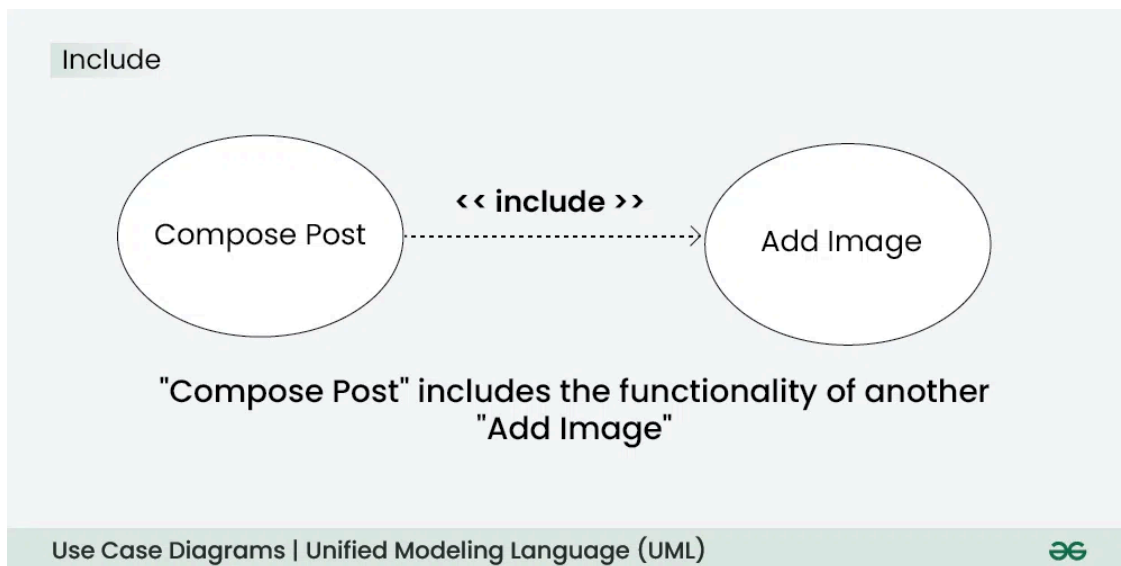


3.2. Include Relationship

The Include Relationship indicates that a use case includes the functionality of another use case. It is denoted by a dashed arrow pointing from the including use case to the included use case. This relationship promotes modular and reusable design.

Example: Social Media Posting

- **Use Cases:** Compose Post, Add Image
- **Include Relationship:** The “Compose Post” use case includes the functionality of “Add Image.” Therefore, composing a post includes the

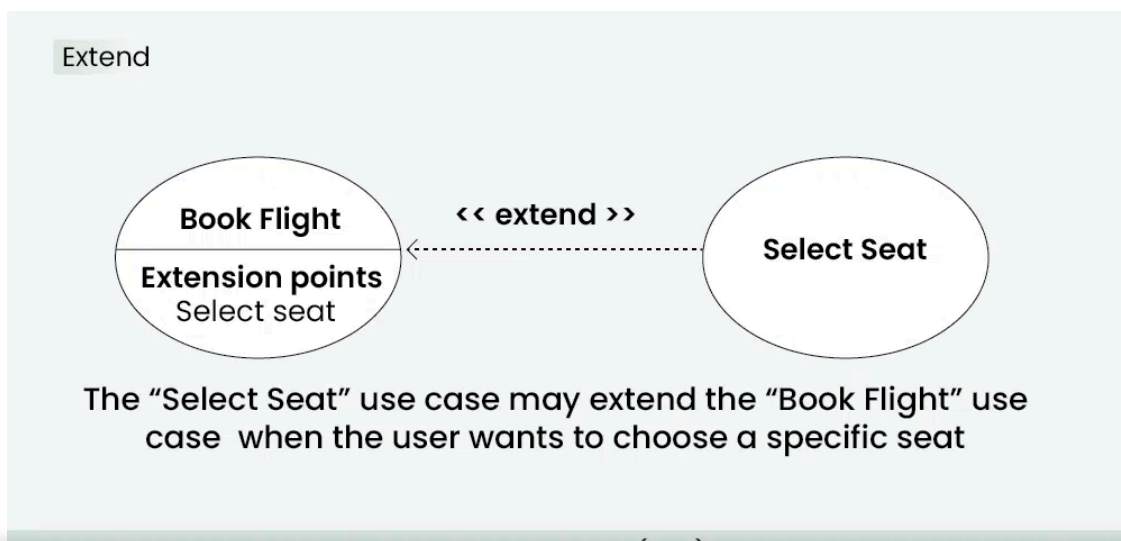


3.3. Extend Relationship

The Extend Relationship illustrates that a use case can be extended by another use case under specific conditions. It is represented by a dashed arrow with the keyword "extend." This relationship is useful for handling optional or exceptional behavior.

Example: Flight Booking System

- **Use Cases:** Book Flight, Select Seat
- **Extend Relationship:** The "Select Seat" use case may extend the "Book Flight" use case when the user wants to choose a specific seat, but it is an optional step.

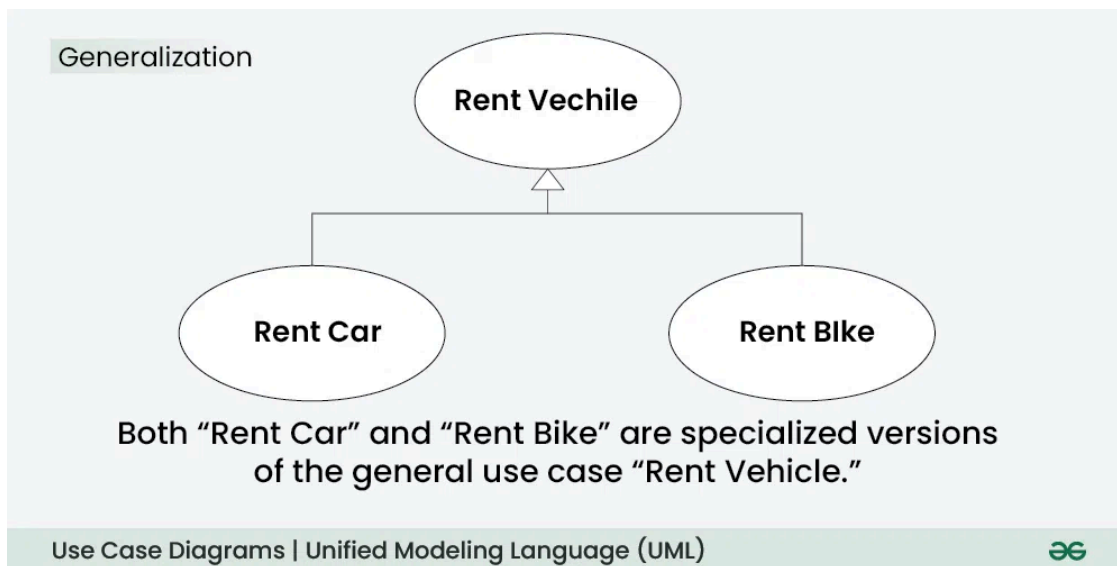


We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

The Generalization Relationship establishes an “is-a” connection between two use cases, indicating that one use case is a specialized version of another. It is represented by an arrow pointing from the specialized use case to the general use case.

Example: Vehicle Rental System

- **Use Cases:** Rent Car, Rent Bike
- **Generalization Relationship:** Both “Rent Car” and “Rent Bike” are specialized versions of the general use case “Rent Vehicle.”



4. How to draw a Use Case diagram in UML?

Step 1: Identify Actors

Determine who or what interacts with the system. These are your actors. They can be users, other systems, or external entities.

Step 2: Identify Use Cases

Identify the main functionalities or actions the system must perform. These are your use cases. Each use case should represent a specific piece of functionality.

Step 3: Connect Actors and Use Cases

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

Draw lines (associations) between actors and the use cases they are involved in. This represents the interactions between actors and the system.

Step 4: Add System Boundary

Draw a box around the actors and use cases to represent the system boundary. This defines the scope of your system.

Step 5: Define Relationships

If certain use cases are related or if one use case is an extension of another, you can indicate these relationships with appropriate notations.

Step 6: Review and Refine

Step back and review your diagram. Ensure that it accurately represents the interactions and relationships in your system. Refine as needed.

Step 7: Validate

Share your use case diagram with stakeholders and gather feedback. Ensure that it aligns with their understanding of the system's functionality.

Let's understand how to draw a Use Case diagram with the help of an Online Shopping System:

1. Actors:

- Customer
- Admin

2. Use Cases:

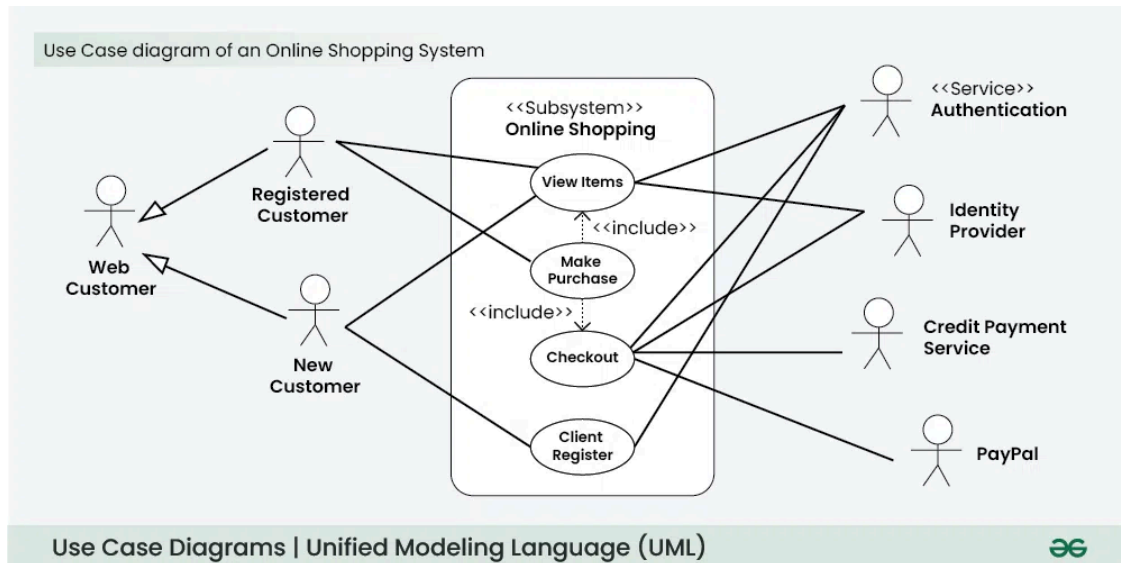
1. Browse Products
2. Add to Cart

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

3. Relations:

- The Customer can browse products, add to the cart, and complete the checkout.
- The Admin can manage the inventory.

Below is the usecase diagram of an Online Shopping System:



5. What are common Use Case Diagram Tools and Platforms?

Several tools and platforms are available to create and design Use Case Diagrams. These tools offer features that simplify the diagram creation process, facilitate collaboration among team members, and enhance overall efficiency. Here are some popular Use Case Diagram tools and platforms:

6.1. Lucidchart

- Cloud-based collaborative platform.
- Intuitive drag-and-drop interface.
- Real-time collaboration and commenting.
- Templates for various diagram types.
- Integration with other tools like Jira and Confluence.

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

- Free, open-source diagramming tool.
- Works offline and can be integrated with Google Drive, Dropbox, and others.
- Offers a wide range of diagram types, including Use Case Diagrams.
- Customizable shapes and themes.

6.3. Microsoft Visio

- Part of the Microsoft Office suite.
- Supports various diagram types, including Use Case Diagrams.
- Integration with Microsoft 365 for collaborative editing.
- Extensive shape libraries and templates.

6.4. SmartDraw

- User-friendly diagramming tool.
- Templates for different types of diagrams, including Use Case Diagrams.
- Integration with Microsoft Office and Google Workspace.
- Auto-formatting and alignment features.

6.5. PlantUML

- Open-source tool for creating UML diagrams.
- Text-based syntax for diagram specification.
- Integrates with various text editors and IDEs.
- Supports collaborative work using version control systems.

6. What are Common Mistakes and Pitfalls while making Use Case Diagram?

Avoiding common mistakes ensures the accuracy and effectiveness of the Use Case Diagram. Here are key points for each mistake:

6.1. Overcomplication:

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

- **Prevention:** Focus on essential use cases and maintain an appropriate level of abstraction.

6.3. Ambiguous Relationships:

- **Mistake:** Unclear relationships between actors and use cases.
- **Impact:** Causes misinterpretation of system interactions.
- **Prevention:** Clearly define and label relationships with proper notation.

6.3. Inconsistent Naming Conventions:

- **Mistake:** Inconsistent naming of actors and use cases.
- **Impact:** Causes confusion and hinders communication.
- **Prevention:** Establish and adhere to a consistent naming convention.

6.4. Misuse of Generalization:

- **Mistake:** Incorrect use of generalization relationships.
- **Impact:** Misrepresentation of the “is-a” relationship between use cases or actors.
- **Prevention:** Ensure accurate usage to represent specialization relationships.

6.5. Overlooking System Boundaries:

- **Mistake:** Not clearly defining the system boundary.
- **Impact:** Challenges understanding of the system’s scope.
- **Prevention:** Clearly enclose relevant actors and use cases within a system boundary.

6.6. Lack of Iteration:

- **Mistake:** Treating the diagram as a static artifact.
- **Impact:** May become outdated and not reflect the current state of the system.
- **Prevention:** Use an iterative approach, updating the diagram as the system

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

7. What can be Use Case Diagram Best Practices?

Creating effective and clear Use Case Diagrams is crucial for communicating system functionality and interactions. Here are some best practices to follow:

7.1 Keep it Simple:

- **Focus on High-Level Functionality:** Avoid unnecessary details and concentrate on representing the system's primary functionalities.
- **Use Concise Language:** Use clear and concise language for use case and actor names to enhance readability.

7.2 Consistency:

- **Naming Conventions:** Maintain a consistent naming convention for use cases and actors throughout the diagram. This promotes clarity and avoids confusion.
- **Formatting Consistency:** Keep a consistent format for elements like ovals (use cases), stick figures (actors), and lines to maintain a professional look.

7.3. Organize and Align:

- **Logical Grouping:** Organize use cases into logical groups to represent different modules or subsystems within the system.
- **Alignment:** Maintain proper alignment of elements to make the diagram visually appealing and easy to follow.

7.4. Use Proper Notation:

- **Consistent Symbols:** Adhere to standard symbols for actors (stick figures), use cases (ovals), and relationships to ensure understanding.
- **Proper Line Types:** Clearly distinguish between association, include, extend, and generalization relationships using appropriate line types.

7.5. Review and Iterate:

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

- **Feedback Loop:** Regularly review the diagram with stakeholders to ensure accuracy and completeness.
- **Iterative Process:** Use an iterative process, updating the diagram as the system evolves or more information becomes available.

By following these best practices, you can create Use Case Diagrams that effectively communicate the essential aspects of a system, fostering a shared understanding among stakeholders and facilitating the development process.

8. What are the Purpose and Benefits of Use Case Diagrams?

The Use Case Diagram offers numerous benefits throughout the system development process. Here are some key advantages of using Use Case Diagrams:

- **Visualization of System Functionality:**
 - Use Case Diagrams provide a visual representation of the system's functionalities and interactions with external entities.
 - This visualization helps stakeholders, including non-technical ones, to understand the system's high-level behavior.
- **Communication:**
 - Use Case Diagrams serve as a powerful communication tool, facilitating discussions between stakeholders, developers, and designers.
 - They provide a common language for discussing system requirements, ensuring a shared understanding among diverse team members.
- **Requirement Analysis:**
 - During the requirements analysis phase, Use Case Diagrams help in identifying, clarifying, and documenting user requirements.
 - They capture the various ways users interact with the system, aiding in a comprehensive understanding of system functionality.
- **Focus on User Goals:**
 - Use Case Diagrams center around user goals and scenarios, emphasizing the perspective of external entities (actors).

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

- **System Design:**
 - In the system design phase, Use Case Diagrams aid in designing how users (actors) will interact with the system.
 - They contribute to the planning of the user interface and help in organizing system functionalities.
- **Testing and Validation:**
 - Use Case Diagrams are valuable for deriving test cases and validating system behavior.
 - Testers can use the diagrams to ensure that all possible scenarios, including alternative and exceptional paths, are considered during testing.

9. Conclusion

In conclusion, a Use Case Diagram in UML serves as a powerful tool for capturing and visualizing the functional requirements and interactions within a system. By representing actors, use cases, and their relationships in a clear and concise manner, this diagram provides a high-level overview of the system's behavior.

Feeling lost in the vast world of System Design? It's time for a transformation! Enroll in our [Mastering System Design](#) From Low-Level to High-Level Solutions - Live Course and embark on an exhilarating journey to efficiently master system design concepts and techniques.

What We Offer:

- Comprehensive Course Coverage
- Expert Guidance for Efficient Learning
- Hands-on Experience with Real-world System Design Project
- Proven Track Record with 100,000+ Successful Enthusiasts

Fault Tolerance in System Design

Message Queues | System Design

Share your thoughts in the comments

Add Your Comment

Similar Reads

Class Diagrams vs Object Diagrams | Unified Modeling Language(UML)

Behavioral Diagrams | Unified Modeling Language(UML)

State Machine Diagrams | Unified Modeling Language (UML)

Activity Diagrams | Unified Modeling Language (UML)

Sequence Diagrams | Unified Modeling Language (UML)

Object Diagrams | Unified Modeling Language (UML)

Structural Diagrams | Unified Modeling Language(UML)

Interaction Overview Diagrams | Unified Modeling Language (UML)

Unified Modeling Language (UML) Diagrams

Collaboration Diagrams | Unified Modeling Language(UML)

S **srinam**

Follow

Article Tags : [Geeks Premier League 2023](#) , [Geeks Premier League](#) , [System Design](#)

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)



A-143, 9th Floor, Sovereign Corporate
Tower, Sector-136, Noida, Uttar Pradesh -
201305



Company

About Us
Legal
Careers
In Media
Contact Us
Advertise with us
GFG Corporate Solution
Placement Training Program

Languages

Python
Java
C++
PHP
GoLang
SQL
R Language
Android Tutorial
Tutorials Archive

Explore

Hack-A-Thons
GfG Weekly Contest
DSA in JAVA/C++
Master System Design
Master CP
GeeksforGeeks Videos
Geeks Community

DSA

Data Structures
Algorithms
DSA for Beginners
Basic DSA Problems
DSA Roadmap
Top 100 DSA Interview Problems
DSA Roadmap by Sandeep Jain
All Cheat Sheets

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

[Data Science With Python](#)[HTML](#)[Data Science For Beginner](#)[CSS](#)[Machine Learning Tutorial](#)[Web Templates](#)[ML Maths](#)[CSS Frameworks](#)[Data Visualisation Tutorial](#)[Bootstrap](#)[Pandas Tutorial](#)[Tailwind CSS](#)[NumPy Tutorial](#)[SASS](#)[NLP Tutorial](#)[LESS](#)[Deep Learning Tutorial](#)[Web Design](#)[Django Tutorial](#)

Python Tutorial

Computer Science

[Python Programming Examples](#)[Operating Systems](#)[Python Projects](#)[Computer Network](#)[Python Tkinter](#)[Database Management System](#)[Web Scraping](#)[Software Engineering](#)[OpenCV Tutorial](#)[Digital Logic Design](#)[Python Interview Question](#)[Engineering Maths](#)

DevOps

Competitive Programming

[Git](#)[Top DS or Algo for CP](#)[AWS](#)[Top 50 Tree](#)[Docker](#)[Top 50 Graph](#)[Kubernetes](#)[Top 50 Array](#)[Azure](#)[Top 50 String](#)[GCP](#)[Top 50 DP](#)[DevOps Roadmap](#)[Top 15 Websites for CP](#)

System Design

JavaScript

[High Level Design](#)[JavaScript Examples](#)[Low Level Design](#)[TypeScript](#)[UML Diagrams](#)[ReactJS](#)[Interview Guide](#)[NextJS](#)

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)

[System Design Bootcamp](#)[Lodash](#)[Interview Questions](#)[Web Browser](#)

Preparation Corner

[Company-Wise Recruitment Process](#)[Resume Templates](#)[Aptitude Preparation](#)[Puzzles](#)[Company-Wise Preparation](#)

School Subjects

[Mathematics](#)[Physics](#)[Chemistry](#)[Biology](#)[Social Science](#)[English Grammar](#)[World GK](#)

Management & Finance

[Management](#)[HR Management](#)[Finance](#)[Income Tax](#)[Organisational Behaviour](#)[Marketing](#)

Free Online Tools

[Typing Test](#)[Image Editor](#)[Code Formatters](#)[Code Converters](#)[Currency Converter](#)[Random Number Generator](#)[Random Password Generator](#)

More Tutorials

[Software Development](#)[Software Testing](#)[Product Management](#)[SAP](#)[SEO - Search Engine Optimization](#)[Linux](#)[Excel](#)

GeeksforGeeks Videos

[DSA](#)[Python](#)[Java](#)[C++](#)[Data Science](#)[CS Subjects](#)

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#)